

LeetCode 930

Binary Subarrays With Sum

© Authored by Arpit Raj, LNMIIT Jaipur | July 6, 2026

$$\text{Exactly(goal)} = \text{AtMost(goal)} - \text{AtMost(goal - 1)}$$

Why Sliding Window Works Here

Normally, the Sliding Window technique doesn't work for arbitrary sums because removing the left element can increase or decrease the sum unpredictably.

But here, the array consists of **0s and 1s only**. This makes the window sum **monotonic**:

- **Expanding** the window can only increase (or keep) the sum.
- **Shrinking** the window can only decrease (or keep) the sum.

That's why sliding window works perfectly for this specific problem!

C++ SOLUTION

```
class Solution {
public:
    int n;

    int atmost(vector<int>& nums, int k) {
        if (k < 0) {
            return 0;
        }
        int ans = 0;
        int sum = 0;
```

```

        int l = 0;
        int r = 0;
        while (r < n) {
            sum += nums[r];
            while (l <= r && sum > k) {
                sum -= nums[l];
                l++;
            }

            ans += r - l + 1;
            r++;
        }
        return ans;
    }

    int numSubarraysWithSum(vector<int>& nums, int goal) {
        n = nums.size();
        return atmost(nums, goal) - atmost(nums, goal - 1);
    }
};

```